SQL SPRINT PROGRAMME - ADVANCED QUERYING

Window Functions in SQL

Ranking, running totals, and row comparisons using OVER(), PARTITION BY, and ORDER BY - with worked SQL examples.

MODULE
Advanced SQL Querying

FORMAT
SQL Theory + Practice

What this report covers

01
OVER() & PARTITION BY
The syntax that powers every window function

02
Ranking Functions
ROW_NUMBER, RANK, DENSE_RANK, NTILE

03
Offset Functions
LAG and LEAD for row-to-row comparison

04
Aggregate Window Functions
Running totals with SUM, AVG OVER()

Prepared by: Riya Kaldate

What is a Window Function?

A window function performs a calculation across a set of rows that are related to the current row - called a 'window' - without collapsing those rows into a single output, unlike GROUP BY. Each row keeps its own identity while still having access to values from other rows in its window.

GROUP BY

Collapses multiple rows into one summary row per group. Individual row detail is lost.

```
5 rows in -> 1 row out
```

Window Function

Keeps every row, but adds a calculated value (rank, running total, etc.) alongside each one.

```
5 rows in -> 5 rows out
```

Sample Table Used in Examples: Sales

emp_id	name	department	sales
1	Aarav	Electronics	5000
2	Priya	Electronics	7000
3	Karan	Clothing	4500
4	Sneha	Clothing	6200
5	Vikram	Electronics	6500

The OVER() Clause

Every window function uses the OVER() clause to define its window - the set of rows it operates on. Two keywords inside OVER() control this: PARTITION BY groups rows (like a 'soft' GROUP BY that doesn't collapse them), and ORDER BY decides the sequence used for ranking, running totals, or row comparisons.

SYNTAX

```
<window_function>(...) OVER (
    PARTITION BY column_name
    ORDER BY column_name
)
```

PARTITION BY

Splits rows into groups; the function resets/restarts for each group. Optional - omit it to treat the whole result set as one window.

ORDER BY

Defines the row sequence within each partition. Required for ranking and offset functions; affects running totals for aggregates.

TIP

Window functions are evaluated AFTER WHERE and GROUP BY, but BEFORE ORDER BY and LIMIT.

Ranking Functions - ROW_NUMBER & RANK

These functions assign a position number to each row within its partition, based on the ORDER BY sequence. They differ only in how they handle ties.

ROW_NUMBER() Always gives a unique, sequential number - even for ties

QUERY.SQL

```
SELECT name, department, sales,
       ROW_NUMBER() OVER (
           PARTITION BY department
           ORDER BY sales DESC) AS row_num
FROM Sales;
```

RANK() Tied rows share the same rank; next rank SKIPS numbers

QUERY.SQL

```
SELECT name, department, sales,
       RANK() OVER (
           PARTITION BY department
           ORDER BY sales DESC) AS sales_rank
FROM Sales;
```

TIP

RANK() with a tie at position 2 jumps straight to 4 next - it does not reuse 3.

DENSE_RANK & NTILE

DENSE_RANK() Tied rows share a rank; next rank does NOT skip

QUERY.SQL

```
SELECT name, sales,
       DENSE_RANK() OVER (ORDER BY sales DESC) AS d_rank
FROM Sales;
```

NTILE(N) Splits rows into n roughly equal-sized buckets

QUERY.SQL

```
SELECT name, sales,
       NTILE(3) OVER (ORDER BY sales DESC) AS sales_tier
FROM Sales;

-- Splits employees into 3 performance tiers
```

Side-by-Side: Sales Sorted Descending (6500 appears twice - a tie)

SALES	ROW_NUMBER	RANK	DENSE_RANK
7000	1	1	1
6500	**2**	**2**	**2**
6500	**3**	**2**	**2**
6200	4	4	3
4500	5	5	4

Offset Functions - LAG & LEAD

LAG and LEAD let a row 'see' a value from a different row in its partition - the previous row or the next row - without needing a self-join. They're commonly used to compare a value against the prior period (e.g. month-over-month change).

LAG() Looks BACKWARD - returns a value from the PREVIOUS row

QUERY.SQL

```
SELECT name, sales,
       LAG(sales, 1) OVER (ORDER BY emp_id) AS prev_sales
FROM Sales;

-- First row's prev_sales is NULL (no row before it)
```

LEAD() Looks FORWARD - returns a value from the NEXT row

QUERY.SQL

```
SELECT name, sales,
       LEAD(sales, 1) OVER (ORDER BY emp_id) AS next_sales
FROM Sales;

-- Last row's next_sales is NULL (no row after it)
```

SYNTAX NOTE

```
LAG(column, offset, default_value)
-- offset and default_value are optional (default offset = 1)
```

Aggregate Window Functions

Familiar aggregate functions - SUM, AVG, MAX, MIN, COUNT - become window functions simply by adding OVER(). This produces running totals, running averages, or group totals shown alongside every individual row, instead of one collapsed result.

EXAMPLE 1 Running total of sales, ordered by employee ID

QUERY.SQL

```
SELECT name, sales,
       SUM(sales) OVER (
           ORDER BY emp_id
           ROWS BETWEEN UNBOUNDED PRECEDING
                    AND CURRENT ROW) AS running_total
FROM Sales;
```

EXAMPLE 2 Compare each employee's sales to their department's total

QUERY.SQL

```
SELECT name, department, sales,
       SUM(sales) OVER (
           PARTITION BY department) AS dept_total
FROM Sales;

-- dept_total repeats the SAME value for every row
-- in that department, unlike GROUP BY which merges rows
```

TIP

Without ORDER BY, SUM() OVER() sums the WHOLE partition for every row - no running effect.

Window Frame Clause - Quick Reference

The frame clause (ROWS BETWEEN ... AND ...) fine-tunes exactly which rows within a partition are included in the calculation, relative to the current row. If omitted, most databases default to RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW when ORDER BY is present.

FRAME KEYWORD	MEANING
`UNBOUNDED PRECEDING`	From the very first row of the partition up to the current row.
`CURRENT ROW`	Just the row being evaluated right now.
`UNBOUNDED FOLLOWING`	From the current row to the very last row of the partition.
`n PRECEDING / n FOLLOWING`	A fixed number of rows before / after the current row (e.g. 3 PRECEDING for a moving average).

EXAMPLE

```
AVG(sales) OVER (
    ORDER BY order_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
)  -- 3-row moving average
```

Quick Reference & Key Takeaways

Ranking Functions

- ROW_NUMBER() - always unique, sequential, no gaps, no shared ranks
- RANK() - ties share a rank; the next rank SKIPS ahead
- DENSE_RANK() - ties share a rank; the next rank does NOT skip
- NTILE(n) - distributes rows into n roughly equal buckets

Offset & Aggregate Functions

- LAG() looks backward to a previous row; LEAD() looks forward to a future row
- SUM/AVG/COUNT + OVER() = running totals or repeated group totals without losing row detail
- PARTITION BY restarts the calculation per group; omitting it treats all rows as one window
- ROWS BETWEEN ... AND ... controls exactly which rows fall inside the calculation frame

One-Line Cheat Sheet

```
<func>(...) OVER (PARTITION BY col ORDER BY col)
```

Why this matters

Window functions let you rank, compare, and accumulate values per row without losing detail or writing complex self-joins - a core skill for analytics, reporting, and leaderboard-style queries.